

Blockchain Experiment 2

Name: Parth Narkar

D20A / 53

AIM: Create a Blockchain using Python

Google Colab Link:

<https://colab.research.google.com/drive/1Fqbae5RWom545vpGhlCE8V9NLLjOXEbq?usp=sharing>

Theory:

Blockchain is a distributed and immutable ledger that records data in a sequence of linked blocks. Each block contains a timestamp, proof (from Proof-of-Work), and the hash of the previous block, ensuring integrity and security.

In this program:

1. **Block Creation** – A block is generated using `create_block()` with a proof and previous hash.
2. **Proof-of-Work (PoW)** – Implemented to mine a block by solving a cryptographic puzzle (finding a hash with leading zeros).
3. **Hashing** – SHA-256 algorithm ensures data immutability.
4. **Validation** – `is_chain_valid()` checks that every block correctly references the previous hash and satisfies PoW conditions.
5. **Flask Web API** – Routes `/mine_block`, `/get_chain`, and `/is_valid` allow mining, fetching the chain, and verifying blockchain integrity through a browser or API client.

This program runs on a local server and simulates mining a blockchain without a network of nodes, making it an ideal introductory model to understand blockchain basics.

Implementation:

```
import datetime
import hashlib
import json
```

```
class Blockchain:
    def __init__(self):
        self.chain = []
        self.create_block(proof = 1, previous_hash = '0')

    def create_block(self, proof, previous_hash):
        block = {'index': len(self.chain) + 1,
                  'timestamp': str(datetime.datetime.now()),
                  'proof': proof,
                  'previous_hash': previous_hash}
        self.chain.append(block)
        return block

    def get_previous_block(self):
        return self.chain[-1]

    def proof_of_work(self, previous_proof):
        new_proof = 1
        check_proof = False
        while check_proof is False:
            hash_operation = hashlib.sha256(str(new_proof**2 - previous_proof**2).encode()).hexdigest()
            if hash_operation[:4] == '0000':
                check_proof = True
            else:
                new_proof += 1
        return new_proof

    def hash(self, block):
        hash_operation = hashlib.sha256(str(block['index'] - previous_proof**2).encode()).hexdigest()
        if hash_operation[:4] != '0000':
            return False
        previous_block = block
        block_index += 1
        return True
```

```
# Creating a Blockchain
blockchain = Blockchain()
```

```
# Mining a new block
def mine_block():
    previous_block = blockchain.get_previous_block()
    previous_proof = previous_block['proof']
    proof = blockchain.proof_of_work(previous_proof)
    previous_hash = blockchain.hash(previous_block)
    block = blockchain.create_block(proof, previous_hash)
    response = {'message': 'Congratulations, you just mined a block!',
                  'index': block['index'],
                  'timestamp': block['timestamp'],
                  'proof': block['proof'],
                  'previous_hash': block['previous_hash']}
    return response
```

```
# Getting the full Blockchain
def get_chain():
```

```
# Getting the full Blockchain
def get_chain():
    response = {'chain': blockchain.chain,
                'length': len(blockchain.chain)}
    return response
```

```
# Checking if the Blockchain is valid
def is_valid():
    is_valid = blockchain.is_chain_valid(blockchain.chain)
    if is_valid:
        response = {'message': 'Parth is a good boy'}
    else:
        response = {'message': 'Parth Builds is on Instagram yayy!'}
    return response
```

```
▶ print("Functions Menu:")
print("=====")
print("1. Mine a block")
print("2. Display the chain")
print("3. Check the validity of the chain")
choice = int(input("Enter your choice :"))
if choice == 1:
    print(mine_block())
elif choice == 2:
    print(get_chain())
elif choice == 3:
    print(is_valid())
```

```
▶ print("Functions Menu:")
print("=====")
print("1. Mine a block")
print("2. Display the chain")
print("3. Check the validity of the chain")
choice = int(input("Enter your choice :"))
if choice == 1:
    print(mine_block())
elif choice == 2:
    print(get_chain())
elif choice == 3:
    print(is_valid())
else:
    print("Invalid choice")
```

... Functions Menu:
=====
1. Mine a block
2. Display the chain
3. Check the validity of the chain
Enter your choice :1
{'message': 'Congratulations, you just mined a block!', 'index': 4, 'timestamp': '2026-08-20 03:28:20.471760', 'proof': 21391, 'previous_hash': '9e495fd34a95378a47ccf236127bc'}

```

▶ print("Functions Menu:")
print("=====")
print("1. Mine a block")
print("2. Display the chain")
print("3. Check the validity of the chain")
choice = int(input("Enter your choice :"))
if choice == 1:
    print(mine_block())
elif choice == 2:
    print(get_chain())
elif choice == 3:
    print(is_valid())
else :
    print("Invalid choice")

```

```

... Functions Menu:
=====
1. Mine a block
2. Display the chain
3. Check the validity of the chain
Enter your choice :2
{'chain': [{'index': 1, 'timestamp': '2026-08-20 03:22:08.692542', 'proof': 1, 'previous_hash': '0'}, {'index': 2, 'timestamp': '2026-08-20 03:23:53.42

```

```

▶ print("Functions Menu:")
print("=====")
print("1. Mine a block")
print("2. Display the chain")
print("3. Check the validity of the chain")
choice = int(input("Enter your choice :"))
if choice == 1:
    print(mine_block())
elif choice == 2:
    print(get_chain())
elif choice == 3:
    print(is_valid())
else :
    print("Invalid choice")

```

```

... Functions Menu:
=====
1. Mine a block
2. Display the chain
3. Check the validity of the chain
Enter your choice :3
{'message': 'Parth is a good boy'}

```

Conclusion:

We successfully created a basic blockchain using Python and Flask that supports block mining, chain retrieval, and validity checks. Proof-of-Work ensures security by making block creation computationally intensive, while cryptographic hashing guarantees immutability. The implementation demonstrates core blockchain principles—decentralization, immutability, and consensus—in a simplified environment, providing a strong foundation for building more advanced blockchain systems.